XLA & PyTorch/XLA

Rapid Onboarding Guide

Theory + Practice · For Engineers with C++/PyTorch Background

---

Part 0 — Environment Setup

Before writing any XLA code you need access to hardware that can run it. This guide uses Kaggle — a free, browser-based platform originally built for data-science competitions, now widely used for ML experimentation. Kaggle provides free TPU and GPU time that would otherwise require a paid cloud account.

> ***Why this matters.*** *TPU stands for Tensor Processing Unit — Google's custom ML accelerator chip (an ASIC: Application-Specific Integrated Circuit). A single Kaggle notebook gives you a v3-8 TPU, which is 8 TPU cores. XLA was designed specifically for this hardware, so it is the most natural place to learn.*

What Kaggle provides

- Free v3-8 TPU VMs (virtual machines) — 8 cores, ~100 GB of HBM (High Bandwidth Memory).
- Pre-installed PyTorch/XLA — no setup commands needed for TPU notebooks.
- A public dataset library — CIFAR-10, Fashion-MNIST, and thousands more are one-click additions.
- Persistent notebooks — your work is saved and can be published so others can fork and run it.
- 30 hours/week of free TPU time and 30 hours/week of free GPU time (as of 2025).

Creating a TPU notebook on Kaggle

- Go to kaggle.com and create a free account.
- Verify your phone number — Kaggle requires this before granting accelerator access.
- Click the blue + Create button in the top-left, then New Notebook.
- In the right-hand Settings panel, find Accelerator and select TPU v3-8.
- The notebook will restart with a TPU VM attached. PyTorch/XLA is already installed.

Exercise 0 Verify the Environment · *15 min*

Confirm the TPU is visible and all 8 cores are accessible:

```
import torch
import torch_xla                            # XLA: the bridge between PyTorch and the XLA compiler
import torch_xla.core.xla_model as xm      # xm: the main user-facing API namespace

print(f"PyTorch version : {torch.__version__}")
print(f"XLA version     : {torch_xla.__version__}")

# xla_device() returns the local accelerator (one TPU core, a GPU, or a CPU)
device  = xm.xla_device()
print(f"Local device    : {device}")

# get_xla_supported_devices lists all visible cores of the given type
devices = xm.get_xla_supported_devices('TPU')
print(f"All TPU cores   : {devices}")   # should show 8 entries on a v3-8
```

If the import fails outside Kaggle, install manually:

```
pip install torch torch_xla[tpu]     # for TPU
pip install torch torch_xla[cuda]    # for GPU
```

Part 1 — What is XLA?

XLA **(Accelerated Linear Algebra)** is a domain-specific compiler for ML (machine learning) workloads — think LLVM **(Low Level Virtual Machine, the compiler infrastructure behind Clang, Rust, and Swift)**, but purpose-built for tensor graphs. Standard PyTorch runs each op immediately (eager mode). XLA instead captures the whole computation, optimises it as a unit, then executes it. This deferred approach enables optimisations that are impossible one op at a time.

Stage	Input → Output	What happens
Frontend	Python model → StableHLO IR	PyTorch/XLA traces the model and emits StableHLO IR (Intermediate Representation — a portable, version-stable graph format; IR is any compiler's internal between-languages representation).
HLO optimisation	StableHLO → optimised HLO	HLO (High-Level Operations) is a DAG (Directed Acyclic Graph — a graph with directed edges and no cycles) of immutable op nodes. Hardware-independent passes: op fusion, constant folding, dead-code elimination, algebraic simplification.
Backend	HLO → native machine code	HLO is lowered to LLVM IR, then to native code. The GPU path uses cuBLAS / cuDNN / Triton for kernels. A buffer-assignment pass allocates all device memory for the whole graph at once — like a register allocator for VRAM (Video RAM, on-device memory).
Runtime	Compiled thunks → execution	PJRT (Plugin JIT Runtime — the uniform device plugin API; JIT = Just-In-Time, compiled at first call then cached) schedules compiled Thunk objects. A Thunk wraps one compiled kernel and issues the actual CUDA/TPU hardware launch.

Three compiler concepts worth knowing

- Fusion — adjacent element-wise ops (add, mul, relu) are merged into a single kernel. One memory round-trip instead of three; directly improves throughput.
- Buffer assignment — XLA sees the entire graph before running anything, so it can prove when two tensors are never live simultaneously and reuse the same VRAM buffer. Eager mode cannot do this.
- Thunks — runtime objects generated per compiled HLO node. The chain HLO → Thunks → kernel launches is the critical path for every training step.

> ***Why this matters.*** *Knowing the pipeline tells you where to look when something goes wrong: slow first step → compilation (HLO size); slow steady state → check fusion; OOM errors → check buffer assignment.*

Exercise 1 First TPU Computation · *30 min*

Observe the two-phase behaviour: lazy graph construction followed by explicit execution.

```
import torch, time
import torch_xla.core.xla_model as xm

device = xm.xla_device()

x = torch.randn(1000, 1000, device=device)
y = torch.randn(1000, 1000, device=device)

t0 = time.time()
z  = torch.matmul(x, y)      # builds the graph — no device work yet
```

```
xm.mark_step()                # compiles + executes the accumulated graph
print(f"First run  (compile + execute) : {time.time()-t0:.3f}s")

t0 = time.time()
z  = torch.matmul(x, y)
xm.mark_step()                # same shape → cache hit, no recompile
print(f"Second run (cache hit)         : {time.time()-t0:.3f}s")
```

Expected: the first run is 5–20× slower than the second. This one-time cost is amortised over thousands of training steps.

Part 2 — Lazy Execution

Standard PyTorch is eager: every op runs immediately and returns a concrete value. PyTorch/XLA is lazy: ops append nodes to an in-memory graph. Nothing runs until mark_step() is called. This deferral is what lets XLA see — and fuse — the whole computation.

Property	Eager (standard PyTorch)	Lazy (PyTorch/XLA)
Execution trigger	Each op individually	xm.mark_step()
Cross-op optimisation	None	Full graph fusion and buffer reuse
Debugging feel	Immediate, intuitive	Errors surface at mark_step()
Performance ceiling	Lower	Higher — especially on TPU

The mark_step() contract

- Call it exactly once per training iteration, after optimizer.step().
- Too few: the graph grows without bound; memory exhausts.
- Too many: the graph is fragmented into small pieces, losing fusion and adding overhead.

> ***Why this matters.*** *mark_step() is the boundary between 'building the graph' and 'running the graph'. Its placement is the single most impactful tuning decision you will make.*

Exercise 2 Observe Lazy Execution · *60 min*

Confirm that no device work happens until mark_step(), and read the metrics report.

```
import torch, torch.nn as nn, time
import torch_xla.core.xla_model as xm
import torch_xla.debug.metrics as met   # met: performance counter API

device = xm.xla_device()
model  = nn.Sequential(nn.Linear(100, 50), nn.ReLU(), nn.Linear(50, 10)).to(device)

# Phase 1: graph tracing — CPU only, no TPU/GPU work
t0 = time.time()
for _ in range(10):
    x = torch.randn(32, 100, device=device)
    y = model(x)
print(f"Tracing (no execution): {time.time()-t0:.4f}s")

# Phase 2: compile + execute
t0 = time.time()
xm.mark_step()
print(f"Compile + execute:      {time.time()-t0:.4f}s")

# Metrics report — look for CompileTime and ExecuteTime entries
```

```
print(met.metrics_report())
```

Eager mode and torch_xla.compile

For debugging, XLA offers full eager mode (immediate execution, no fusion). For production, torch_xla.compile() JIT-compiles one specific function while leaving the rest of the code in eager mode — the best of both worlds.

```
import torch_xla

# Option A — full eager mode (debugging only; no fusion)
torch_xla.experimental.eager_mode(True)

# Option B — compile one hot path, keep everything else eager (recommended)
compiled_model = torch_xla.compile(model)
y = compiled_model(x)   # first call compiles and caches; subsequent calls reuse
```

Exercise 3 Benchmark Eager vs Compiled *· 45 min*

Measure throughput across 100 forward passes and compute the speedup ratio:

```
torch_xla.experimental.eager_mode(True)
device = xm.xla_device()

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 10)
    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

model    = Net().to(device)
compiled = torch_xla.compile(model)
x        = torch.randn(64, 784, device=device)

t0 = time.time()
for _ in range(100): model(x)
t_eager = time.time() - t0

t0 = time.time()
for _ in range(100): compiled(x)
t_comp = time.time() - t0

print(f"Eager: {t_eager:.3f}s   Compiled: {t_comp:.3f}s   Speedup: {t_eager/t_comp:.2f}x")
```

On a Kaggle v3-8 TPU expect a 3–10× speedup. The gap widens with larger models.

Part 3 — Porting a Training Loop to XLA

Migrating from standard PyTorch requires exactly three changes: (1) swap the device, (2) add mark_step(), (3) log infrequently. Model definitions, loss functions, and optimisers are unchanged.

> ***Why this matters.*** *XLA's lazy model means that loss.item() — which forces a device-to-host transfer and stalls the graph — inside the loop breaks execution into one tiny fragment per step. Move all logging outside the hot path.*

Exercise 4 Port a Standard Training Loop *· 2 hours*

The three required changes are marked in the XLA version below.

```
# ── BEFORE: standard PyTorch ──────────────────────────────────────────
import torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

device  = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model   = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10)).to(device)
dataset = TensorDataset(torch.randn(1000, 784), torch.randint(0, 10, (1000,)))
loader  = DataLoader(dataset, batch_size=32, shuffle=True)
opt     = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(5):
    for step, (data, target) in enumerate(loader):
        data, target = data.to(device), target.to(device)
        opt.zero_grad()
        loss_fn(model(data), target).backward()
        opt.step()
        if step % 10 == 0:
            print(loss_fn(model(data), target).item())   # transfer every 10 steps

# ── AFTER: PyTorch/XLA ────────────────────────────────────────────────
import torch_xla.core.xla_model as xm

device  = xm.xla_device()                                # CHANGE 1: XLA device
model   = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10)).to(device)
dataset = TensorDataset(torch.randn(1000, 784), torch.randint(0, 10, (1000,)))
loader  = DataLoader(dataset, batch_size=32, shuffle=True,
                     drop_last=True)    # drop_last: discard the final partial batch
opt     = torch.optim.Adam(model.parameters(), lr=1e-3)  #   to keep shapes constant
loss_fn = nn.CrossEntropyLoss()

for epoch in range(5):
    for step, (data, target) in enumerate(loader):
        data, target = data.to(device), target.to(device)
        opt.zero_grad()
        loss = loss_fn(model(data), target)
        loss.backward()
        opt.step()
        xm.mark_step()                                    # CHANGE 2: execute graph
        if step % 100 == 0:                               # CHANGE 3: log infrequently
            print(loss.cpu().item())                      #   .cpu() before .item()
```

Host-device transfer rules

Pattern	Cost	Fix
loss.item() every step	Stalls and fragments the graph each step	Log every 100+ steps
print(tensor) inside the loop	Implicit transfer and graph stall	Use xm.add_step_closure() for async logging
Python if/for on an XLA tensor	Breaks the graph at the branch point	Use xm.while_loop or scan

Exercise 5 Profile Your Training Loop · *90 min*

After porting, check that the graph is executing as expected. Three counters matter most:

- CompileTime — should appear once or twice total, not once per step.
- ExecuteTime — should dominate wall time in a well-written loop.
- TransferFromDeviceTime — should be near zero; any significant value signals excess .item() calls.

```
import torch_xla.debug.metrics as met
import torch_xla.debug.profiler as xp   # xp: XProf profiler interface

server = xp.start_server(9012)          # TensorBoard XProf can connect at localhost:9012
```

```
# ... your training loop here ...

print(met.metrics_report())             # prints all counters after the run
```

Part 4 — Recompilation: The Silent Performance Killer

XLA caches compiled programs keyed on the exact set of tensor shapes in each forward pass. If any shape changes between steps — even by one element — XLA must compile an entirely new program. On a TPU, compilation takes seconds; across a 10,000-step run this can easily cost hours.

Cause	Why it recompiles	Fix
Variable batch size	Last batch smaller than the rest	drop_last=True on DataLoader
Dynamic sequence length	Different input length per sample	Pad all sequences to a fixed max length
Python branches on tensor data	Control flow alters the graph structure	Use XLA control flow: xm.while_loop, scan

> ***Why this matters.*** *One recompilation per epoch is acceptable — that cost is amortised. One per step is catastrophic. met.metrics_report() will tell you which is happening; CompileTime appearing many times is the tell.*

Exercise 6 Trigger and Count Recompilations · *90 min*

Use the metrics API to contrast a dynamic-shape loop against a fixed-shape loop:

```
import torch, torch.nn as nn
import torch_xla.core.xla_model as xm
import torch_xla.debug.metrics as met

device = xm.xla_device()
model  = nn.Linear(100, 10).to(device)

def run(batch_sizes: list) -> None:
    met.clear_all()
    for bs in batch_sizes:
        model(torch.randn(bs, 100, device=device))
        xm.mark_step()
    data = met.metric_data("CompileTime")
    n    = len(data[1]) if data else 0
    print(f"Batches {batch_sizes}  ->  compilations: {n}")

run([32, 64, 32, 64, 32])   # BAD  — new shape each step = new compile each step
run([32, 32, 32, 32, 32])   # GOOD — same shape each step = compile once, then cache
```

Exercise 7 Dump and Inspect HLO Output · *60 min*

Read the compiled graph directly to see what XLA actually produced:

```
import os, torch, torch.nn as nn
import torch_xla.core.xla_model as xm

# XLA_FLAGS: an environment variable that passes options to the XLA compiler
os.environ['XLA_FLAGS'] = '--xla_dump_to=/tmp/xla_dump'

device = xm.xla_device()
model  = nn.Sequential(nn.Linear(10, 5), nn.ReLU(), nn.Linear(5, 2)).to(device)
model(torch.randn(4, 10, device=device))
xm.mark_step()
```

```
# In a shell / notebook cell:
# !ls -lh /tmp/xla_dump/
#
# Two key files per compilation:
#   *.before_optimizations.txt  — raw HLO before fusion passes
#   *.after_optimizations.txt   — fused HLO actually sent to hardware
#
# Comparing them shows exactly which ops were merged into a single kernel.
```

Part 5 — Distributed Data Parallel (DDP)

DDP **(Distributed Data Parallel)** replicates the model across multiple devices. Each device processes a different shard of each batch. After the backward pass, gradients are all-reduced (summed across all replicas and divided by replica count) so every device's weights remain identical. PyTorch/XLA supports the standard `torch.nn.parallel.DistributedDataParallel` API with an XLA-specific backend that integrates the all-reduce into the compiled graph.

Aspect	Standard DDP (GPU)	XLA DDP
Backend	nccl or gloo	xla
Gradient sync	During backward()	At mark_step()
Launcher	torchrun / mp.spawn()	torch_xla.launch()
Rank query	dist.get_rank()	xr.global_ordinal()
World size	dist.get_world_size()	xr.world_size()

> ***Why this matters.*** *nccl (NVIDIA Collective Communications Library) is the standard GPU all-reduce backend. XLA replaces it with its own collective, which is baked into the compiled program — no separate blocking synchronisation call.*

Minimal DDP template

```
import torch, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
import torch_xla, torch_xla.core.xla_model as xm
import torch_xla.runtime as xr             # xr: runtime helpers (rank, world size)
import torch_xla.distributed.xla_backend   # registers 'xla' as a valid process-group backend

def train_fn(rank: int) -> None:
    # 1. Initialise the distributed process group
    dist.init_process_group("xla", rank=rank, world_size=xr.world_size())

    device    = xm.xla_device()
    model     = MyModel().to(device)

    # 2. Wrap with DDP.
    #    gradient_as_bucket_view=True is REQUIRED — without it, XLA cannot
    #    fuse the all-reduce into the compiled graph, silently breaking sync.
    ddp_model = DDP(model, gradient_as_bucket_view=True)
    opt       = torch.optim.Adam(ddp_model.parameters())

    for data, target in dataloader:
        data, target = data.to(device), target.to(device)
        opt.zero_grad()
        loss_fn(ddp_model(data), target).backward()
        opt.step()
        xm.mark_step()   # all-reduce + weight update compiled into one graph execution

    dist.destroy_process_group()
```

```
# torch_xla.launch() spawns one process per visible XLA device automatically
torch_xla.launch(train_fn)
```

- gradient_as_bucket_view=True is mandatory. Omitting it breaks gradient sync silently — no error, wrong results.
- torch_xla.launch() is the XLA-aware launcher. mp.spawn() is the standard Python multiprocessing launcher and is unaware of XLA device topology; do not use it here.
- xr.global_ordinal() returns this process's rank; xr.world_size() the total replica count.

Part 6 — Performance Optimisation

AMP — Automatic Mixed Precision

AMP **(Automatic Mixed Precision)** runs selected ops in a reduced-precision float format — **bfloat16** (Brain Float 16: a 16-bit float format with the same exponent range as float32, but fewer mantissa bits) on TPU, or float16 on GPU. This roughly halves memory use and can double throughput on hardware with native 16-bit units, at negligible accuracy cost for most models.

PyTorch/XLA provides its own autocast context and GradScaler (which rescales loss to prevent float16 gradient underflow). On TPUs, bfloat16 is natively supported and does not need loss scaling, but the API is identical.

```
from torch_xla.amp import autocast, GradScaler

scaler = GradScaler()

for inputs, labels in loader:
    inputs, labels = inputs.to(device), labels.to(device)
    with autocast():                        # ops inside run in bfloat16 / float16
        outputs = model(inputs)
        loss    = criterion(outputs, labels)
    scaler.scale(loss).backward()           # scale prevents float16 gradient underflow
    scaler.step(opt)
    scaler.update()
    xm.mark_step()
```

Exercise 8 ResNet-18 Benchmark: CPU vs TPU *· 90 min*

ResNet-18 is a standard 18-layer residual convolutional network, widely used as a benchmark: large enough to be representative, small enough to train quickly. Residual connections (skip connections that add the input of a block to its output) prevent gradient vanishing in deep networks. This exercise quantifies the real-world speedup XLA delivers.

```
import time, torch, torch.nn as nn
import torchvision.models as models        # torchvision: standard vision model library
import torch_xla.core.xla_model as xm
from torch.utils.data import DataLoader, TensorDataset

def benchmark(device, epochs=3, batch_size=32):
    model   = models.resnet18(num_classes=10).to(device)
    opt     = torch.optim.SGD(model.parameters(), lr=0.01)
    loss_fn = nn.CrossEntropyLoss()
    # Synthetic CIFAR-10-sized data: 3-channel 224x224 images, 10 classes
    loader  = DataLoader(
        TensorDataset(torch.randn(500, 3, 224, 224), torch.randint(0, 10, (500,))),
        batch_size=batch_size, drop_last=True
    )
    t0 = time.time()
    for _ in range(epochs):
        for x, y in loader:
            x, y = x.to(device), y.to(device)
```

```
            opt.zero_grad()
            loss_fn(model(x), y).backward()
            opt.step()
            if 'xla' in str(device):
                xm.mark_step()
    return time.time() - t0

cpu_t = benchmark('cpu')
tpu_t = benchmark(xm.xla_device())
print(f"CPU: {cpu_t:.1f}s   TPU: {tpu_t:.1f}s   Speedup: {cpu_t/tpu_t:.2f}x")
```

Typical result on Kaggle v3-8: 8–20× speedup over a single CPU core.

Part 7 — Capstone Project

Build a complete, optimised training pipeline on a real dataset, published as a Kaggle notebook. This forces every concept in this guide to be applied under realistic conditions, and gives you a permanent reference you can fork for future projects.

Publishing on Kaggle

A Kaggle notebook is a Jupyter notebook (browser-based Python environment with cells you run individually) that executes on Kaggle's servers with your chosen accelerator attached. Notebooks can be made public — click the Share button and set visibility to Public. Anyone can then fork your notebook (Copy & Edit), run it on their own TPU allocation, and build on it. Publishing is optional but is good ML engineering practice: it creates a reproducible record of your work.

Requirements

- A real Kaggle dataset — CIFAR-10 and Fashion-MNIST are one-click additions via the Data panel in the notebook editor.
- DataLoader with drop_last=True and a fixed batch size throughout.
- Model trained in lazy mode; then repeated with eager + torch_xla.compile to compare.
- AMP enabled via autocast + GradScaler.
- Loss logged every 100 steps via xm.add_step_closure() — not via .item() inside the loop.
- Final met.metrics_report() printed, with commentary on any bottlenecks found.
- A printed table comparing CPU training time vs TPU training time.

Suggested models

- EfficientNet-B0 — efficient image classifier; fast on TPU and a good first choice.
- BERT-base — transformer (attention-based architecture) for text classification; pad all sequences to a fixed length to avoid recompilation.
- U-Net — encoder-decoder architecture for image segmentation; tests more complex graph shapes.

> ***Why this matters.*** *Publishing your notebook on Kaggle lets others verify your benchmarks. Reproducibility is a foundational habit in ML engineering — if someone else cannot reproduce your numbers, they cannot trust them.*

Appendix — Quick Reference

Core API

Call	What it does
xm.xla_device()	Return the local XLA device handle
xm.mark_step()	Compile and execute the accumulated graph
xr.global_ordinal()	This process's rank in the DDP group
xr.world_size()	Total number of DDP processes
xm.all_reduce()	Collective reduce across all devices (sum, mean, …)
xm.all_gather()	Gather tensors from every device into one
met.metrics_report()	Print all performance counters
met.clear_all()	Reset counters before benchmarking a section
torch_xla.launch(fn)	Spawn one process per XLA device; call fn(rank)
torch_xla.compile(fn)	JIT-compile fn with XLA; cache on shape

Environment variables

Variable	Effect
PJRT_DEVICE=TPU|CUDA|CPU	Select the device backend
XLA_FLAGS=--xla_dump_to=/path	Write HLO files for every compilation
XLA_USE_BF16=1	Force all float32 ops to bfloat16 on TPU
PT_XLA_DEBUG=1	Enable verbose debug logging to stderr

Common pitfalls

Symptom	Root cause	Fix
Memory grows without bound	No mark_step() call	Add xm.mark_step() after opt.step()
Step time grows, then OOM	Too many mark_step() calls	One call per training step
Many CompileTime counter entries	Dynamic tensor shapes	drop_last=True; pad sequences to fixed length
DDP gradients diverge silently	Missing gradient_as_bucket_view=True	Add it to DDP(...)
TPU hangs on data loading	num_workers > 0 on TPU	Set num_workers=0
Unexpected slowdown after a change	CPU and XLA tensors mixed	Audit all .to(device) calls

Abbreviation glossary

Term	Expansion	Meaning
XLA	Accelerated Linear Algebra	Google's ML-specific compiler
HLO	High-Level Operations	XLA's internal graph IR — a DAG of immutable op nodes
IR	Intermediate Representation	A compiler's internal graph format, between source code and machine code
DAG	Directed Acyclic Graph	A graph with directed edges and no cycles — the shape of any computation graph
PJRT	Plugin JIT Runtime	XLA's uniform device plugin API; abstracts TPU, GPU, and CPU
JIT	Just-In-Time (compilation)	Compiled at first call, cached for all subsequent calls

Term	Expansion	Meaning
TPU	Tensor Processing Unit	Google's custom ML accelerator ASIC
ASIC	Application-Specific Integrated Circuit	A chip designed for exactly one task — faster and more efficient than a general CPU
DDP	Distributed Data Parallel	Training strategy: one model replica per device, gradients all-reduced after each step
AMP	Automatic Mixed Precision	Running ops in float16 or bfloat16 to save memory and increase throughput
BF16	Brain Float 16	16-bit float with float32's exponent range; preferred on TPU; less rounding error than float16
VRAM	Video RAM	On-device memory on a GPU or TPU core
LLVM	Low Level Virtual Machine	Compiler infrastructure used by Clang, Rust, and Swift — and XLA's backend
nccl	NVIDIA Collective Communications Library	Standard GPU all-reduce backend; replaced by 'xla' backend in PyTorch/XLA
HBM	High Bandwidth Memory	The fast on-package memory used by TPUs and high-end GPUs
OOM	Out Of Memory	Device memory exhaustion — often caused by missing mark_step() or too-large batch sizes
VM	Virtual Machine	A software-emulated computer; Kaggle runs your notebook on a VM with a real TPU attached
Kaggle	—	A free cloud platform for ML; provides TPU/GPU VMs and public datasets at no cost

Resources

- XLA architecture: https://openxla.org/xla/architecture
- PyTorch/XLA docs: https://docs.pytorch.org/xla/
- DDP guide: https://docs.pytorch.org/xla/master/perf/ddp.html
- Performance guide: github.com/pytorch/xla/blob/master/PERFORMANCE_GUIDE.md
- Kaggle TPU notebooks: github.com/pytorch/xla/tree/master/contrib/kaggle
- Kaggle signup: https://kaggle.com (free account; phone verification required for TPU)